

Apache Iceberg

BigQuery Metastore Catalog Module

Architecture Design Document

Module: iceberg-bigquery
Version: Apache Iceberg 1.9.2
Package: org.apache.iceberg.gcp.bigquery
Date: 2026-03-24

Contents

- 1. Module Overview
- 2. Architecture Design
 - 2.1 Layered Architecture
 - 2.2 Core Class Hierarchy
 - 2.3 Architecture Diagram
- 3. Design Patterns
 - 3.1 Template Method Pattern
 - 3.2 Strategy Pattern
 - 3.3 Optimistic Concurrency Control
 - 3.4 Builder / Factory
 - 3.5 Retry with Exponential Backoff
- 4. Core Flow Diagrams
 - 4.1 Catalog Initialization
 - 4.2 Table Commit Flow
 - 4.3 Table Load / Refresh
- 5. Sequence Diagrams
 - 5.1 Create Table
 - 5.2 Load Table
 - 5.3 Update Table
- 6. Key Invocation Chains
- 7. Configuration
- 8. Error Handling
- 9. Summary

1. Module Overview

Apache Iceberg BigQuery module (iceberg-bigquery) is an integration bridge between the Apache Iceberg table format and Google Cloud BigQuery Metastore. It implements the Iceberg Catalog SPI, enabling compute engines like Spark, Flink, Trino, and Presto to manage Iceberg tables whose metadata is stored in BigQuery Metastore.

The module adopts a clean layered architecture with only 6 source files, demonstrating how Iceberg's extensible design allows seamless integration with new metastores.

Core Capabilities

- Catalog Management - Full CRUD for namespaces (BigQuery Datasets) and tables
- Metadata Operations - Atomic commits via ETag-based optimistic concurrency
- Authentication - Google ADC + Service Account Impersonation
- Retry Resilience - Built-in retry with BigQueryRetryHelper
- HMS Compatibility - Hive Metastore-compatible parameters for cross-engine interop

Module File Structure

Class	Role	Lines
BigQueryMetastoreCatalog	Catalog entry point, namespace + table CRUD	362
BigQueryMetastoreClient	Client interface (SPI)	128
BigQueryMetastoreClientImpl	REST client with retry + error mapping	682
BigQueryTableOperations	Table metadata refresh + commit	245
BigQueryProperties	Config parsing + credential building	198
BigQueryMetastoreUtils	ExternalCatalog options factory	76

2. Architecture Design

2.1 Layered Architecture

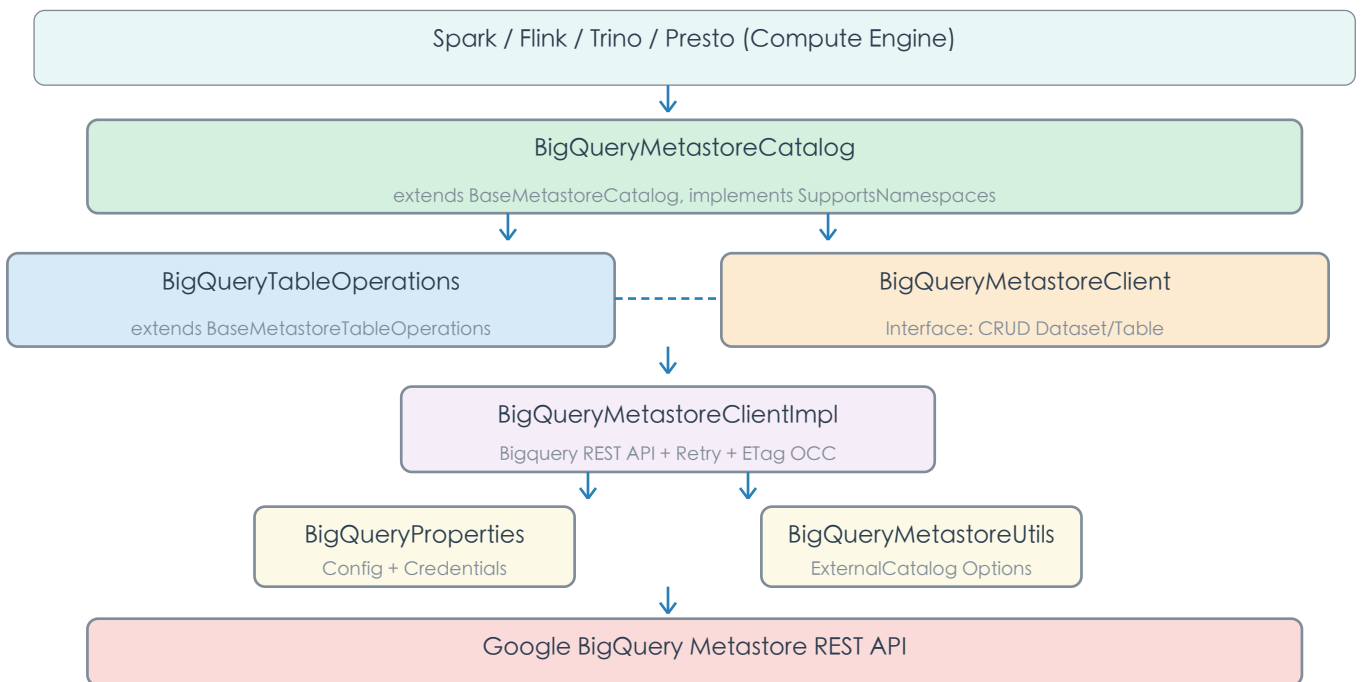
The module follows a strict four-layer architecture separating concerns clearly:

Layer	Class(es)	Responsibility
Catalog	BigQueryMetastoreCatalog	Initialize, namespace/table lifecycle
Table Operations	BigQueryTableOperations	Metadata refresh, atomic commits with OCC
Client	Client + ClientImpl	BigQuery REST API, retry, error translation
Infrastructure	Properties + Utils	Config, credentials, option objects

2.2 Core Class Hierarchy

Iceberg SPI	BigQuery Implementation	Key Methods
Catalog	BigQueryMetastoreCatalog	initialize(), listTables(), dropTable()
BaseMetastoreCatalog	(parent class)	newTableOps(), defaultWarehouseLocation()
SupportsNamespaces	(interface)	createNamespace(), listNamespaces()
TableOperations	BigQueryTableOperations	doRefresh(), doCommit(), io()
BaseMetastoreTableOps	(parent class)	commit(), refresh(), writeNewMetadata()

2.3 Architecture Diagram



3. Design Patterns

3.1 Template Method Pattern

The Template Method pattern is foundational in Iceberg's Catalog SPI. Base classes define the algorithm skeleton while subclasses fill in metastore-specific steps.

`BaseMetastoreCatalog.loadTable()` calls `newTableOps()` (abstract, overridden by `BigQueryMetastoreCatalog` to return `BigQueryTableOperations`), then wraps the result in `BaseTable`. `BaseMetastoreTableOperations.commit()` performs stale-check, calls `doCommit()` (abstract, implemented by `BigQueryTableOperations`), then cleans up old metadata files.

Template (Base Class)	Hook Method	BigQuery Implementation
<code>BaseMetastoreCatalog.loadTable()</code>	<code>newTableOps()</code>	Returns <code>BigQueryTableOperations</code>
<code>BaseMetastoreCatalog.buildTable()</code>	<code>defaultWarehouseLocation()</code>	Uses Dataset storage URI
<code>BaseMetastoreTableOps.commit()</code>	<code>doCommit()</code>	<code>createTable()</code> / <code>updateTable()</code>
<code>BaseMetastoreTableOps.refresh()</code>	<code>doRefresh()</code>	<code>client.load()</code> + parse metadata

3.2 Strategy Pattern (Client Abstraction)

`BigQueryMetastoreClient` interface defines the contract for all operations. The concrete implementation can be swapped for testing - `FakeBigQueryMetastoreClient` validates catalog behavior without real GCP calls. The `@VisibleForTesting initialize()` overload enables dependency injection.

3.3 Optimistic Concurrency Control (OCC)

Iceberg requires atomic metadata commits. BigQuery module uses ETag-based OCC:

- `doRefresh()`: Load table from BigQuery, cache Table object (with ETag) in volatile `metastoreTable`
- `doCommit()` - `updateTable()`: Send HTTP PATCH with `If-Match: {etag}` header
- HTTP 412 Precondition Failed -> `CommitFailedException` -> retry read-modify-write

[Design insight] The volatile Table `metastoreTable` field bridges `doRefresh()` and `doCommit()`, avoiding redundant API calls while ensuring latest ETag is used.

3.4 Builder Pattern + Factory Method

`BigQueryMetastoreUtils` acts as Factory for `ExternalCatalogTableOptions` with nested `StorageDescriptor` and `SerdeInfo` using Hive-compatible constants.

`BigQueryProperties` parses `Map<String,String>`, validates, and constructs `BigQueryOptions` with correct credentials (ADC or Impersonated).

3.5 Retry with Exponential Backoff

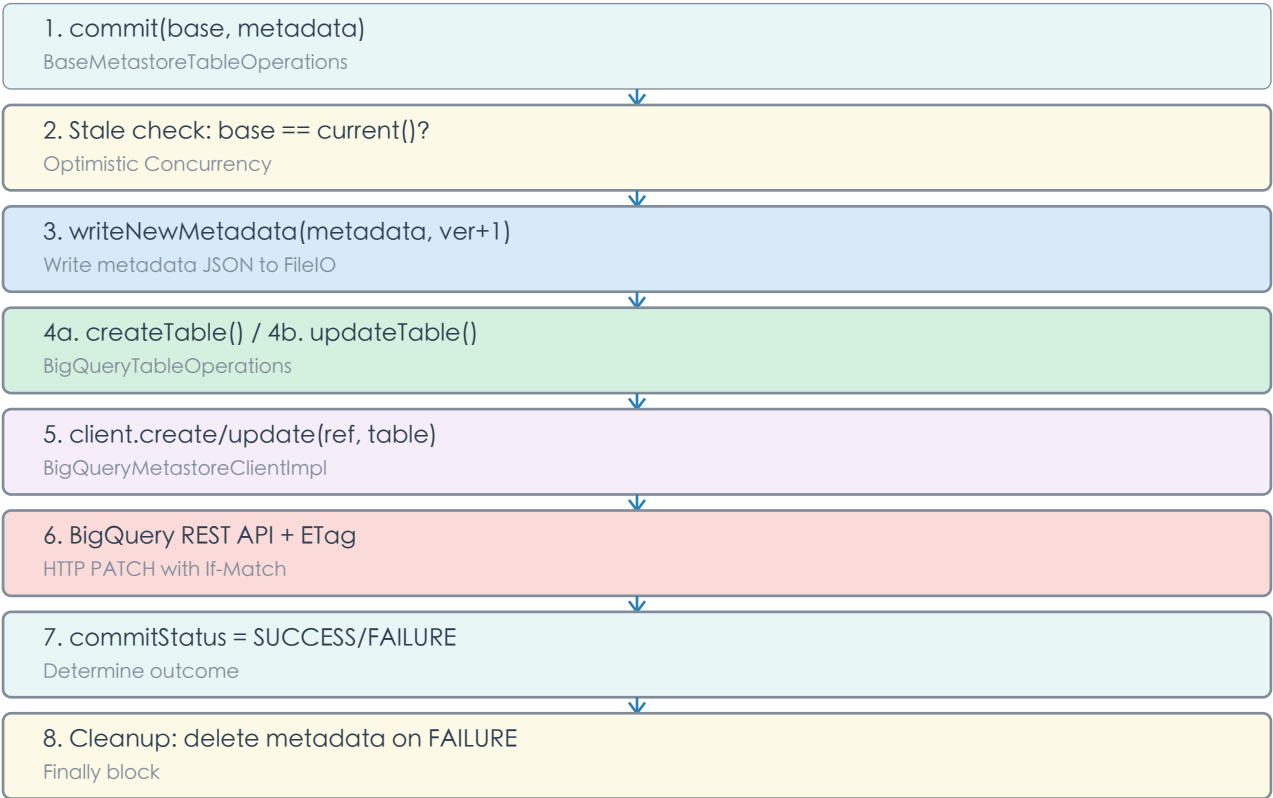
All mutating operations wrapped with `BigQueryRetryHelper.runWithRetries()`. Retryable: rate limit, `ConnectException`, `UnknownHostException`. Non-retryable: `RuntimeException`. Custom `EXCEPTION_HANDLER_INTERCEPTOR` checks `BaseServiceException.isRetryable()`.

4. Core Flow Diagrams

4.1 Catalog Initialization Flow

Step	Operation	Detail
1	BigQueryProperties(properties)	Parse project-id, location, impersonation settings
2	metastoreOptions()	Build BigQueryOptions with credentials
3	new BigQueryMetastoreClientImpl(opt)	Create REST client with GoogleNetHttpTransport
4	Set warehouseLocation	From CatalogProperties.WAREHOUSE_LOCATION
5	CatalogUtil.loadFileIO()	Initialize FileIO (default: ResolvingFileIO)

4.2 Table Commit Flow (doCommit)

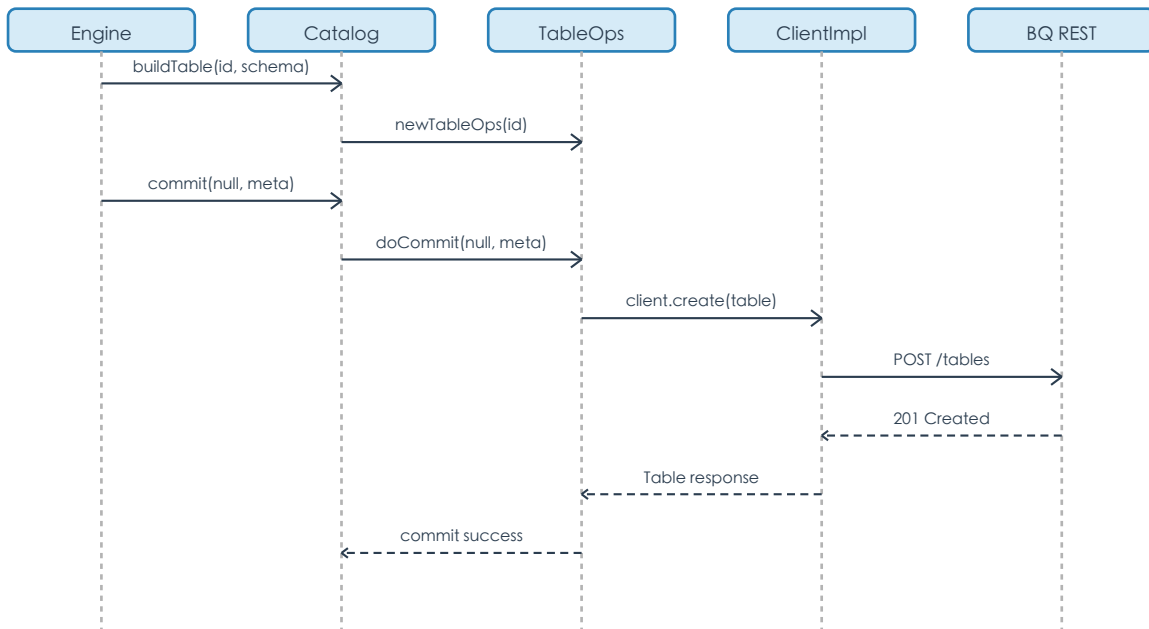


4.3 Table Load / Refresh Flow

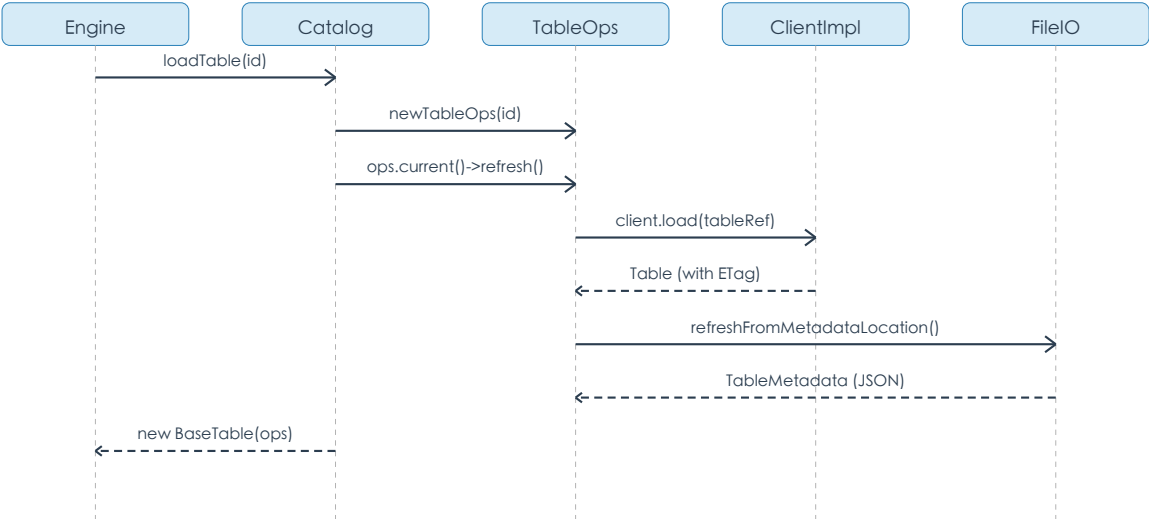
#	Component	Operation
1	BaseMetastoreCatalog	loadTable(id) -> newTableOps(id)
2	BigQueryMetastoreCatalog	new BigQueryTableOperations(client, fileIO, ref)
3	BaseMetastoreTableOps	current() -> refresh() -> doRefresh()
4	BigQueryTableOperations	doRefresh() -> client.load(tableReference)
5	BigQueryMetastoreClientImpl	HTTP GET /tables/{project}/{dataset}/{table}
6	BigQueryTableOperations	validateTable() -> isValidIcebergTable()
7	BigQueryTableOperations	loadMetadataLocationOrThrow() -> metadata_location
8	BaseMetastoreTableOps	refreshFromMetadataLocation() -> read JSON
9	BaseMetastoreCatalog	new BaseTable(ops, fullName, metricsReporter)

5. Sequence Diagrams

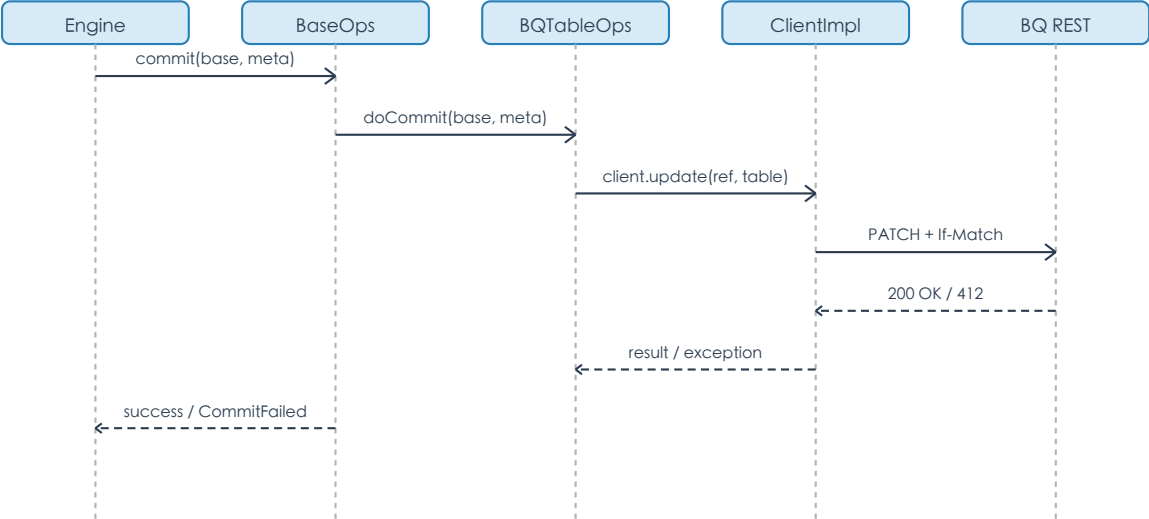
5.1 Create Table Sequence



5.2 Load Table Sequence



5.3 Update Table (Commit) Sequence



6. Key Invocation Chains

6.1 Create Table Chain

```
Engine: catalog.buildTable(id, schema).create()
-> BaseMetastoreCatalog.CatalogTableBuilder.create()
-> ops = newTableOps(id) [BigQueryMetastoreCatalog]
-> new BigQueryTableOperations(client, fileIO, tableRef)
-> ops.commit(null, metadata) [BaseMetastoreTableOperations]
-> doCommit(null, metadata) [BigQueryTableOperations]
-> writeNewMetadata(metadata, 1) -> FileIO writes JSON
-> createTable(newMetaLoc, metadata)
-> makeNewTable() -> createExternalCatalogTableOptions()
-> buildTableParameters() -> metadata_location, type, uuid
-> client.create(table) -> runWithRetries(internalCreate)
-> HTTP POST /tables/{project}/{dataset}
```

6.2 Load Table Chain

```
Engine: catalog.loadTable(identifier)
-> BaseMetastoreCatalog.loadTable(identifier)
-> isValidIdentifier(id) -> validateNamespace(1-level only)
-> ops = newTableOps(id) [BigQueryMetastoreCatalog]
-> ops.current() -> shouldRefresh=true -> refresh()
-> doRefresh() [BigQueryTableOperations]
-> client.load(tableRef) -> HTTP GET + validateTable()
-> cache Table in volatile metastoreTable field
-> loadMetadataLocationOrThrow(options)
-> refreshFromMetadataLocation(loc) -> FileIO read JSON
-> new BaseTable(ops, fullTableName, metricsReporter)
```

6.3 Drop Table Chain

```
Engine: catalog.dropTable(identifier, purge)
-> BigQueryMetastoreCatalog.dropTable(id, purge)
-> ops = newTableOps(id); lastMetadata = ops.current()
-> client.delete(tableRef) -> load() then HTTP DELETE
-> if purge: CatalogUtil.dropTableData(io, lastMetadata)
```

6.4 Namespace Operations

```
createNamespace(ns, metadata):
-> validateNamespace -> new Dataset -> createExternalCatalogDatasetOptions()
-> client.create(dataset) -> runWithRetries -> POST /datasets

listNamespaces():
-> client.list(projectId) -> paginated GET /datasets
-> map Datasets -> Namespace.of(datasetId)

setProperty(ns, props):
-> client.load(dsRef) -> merge params -> runWithRetries(update)
```

7. Configuration Properties

Property Key	Req	Default	Description
gcp.bigquery.project-id	Yes	-	GCP project ID
gcp.bigquery.location	No	us	GCP region/location
gcp.bigquery.list-all-tables	No	true	Filter non-Iceberg on list
gcp.bigquery.impersonate.service-account	No	-	Target SA for impersonation
gcp.bigquery.impersonate.lifetime-seconds	No	3600	Token lifetime (seconds)
gcp.bigquery.impersonate.scopes	No	cloud-platform	Auth scopes
gcp.bigquery.impersonate.delegates	No	-	Delegate chain
warehouse	Cond	-	Default storage base URI

[Credential Resolution] If impersonate.service-account is set, creates ImpersonatedCredentials from ADC as source. Otherwise uses ADC scoped to BigqueryScopes.all().

8. Error Handling & Exception Mapping

BigQueryMetastoreClientImpl has centralized error mapping via convertExceptionIfUnsuccessful():

HTTP Status	Code	Iceberg Exception	Semantics
401 Unauthorized	401	NotAuthorizedException	Invalid credentials
400 Bad Request	400	BadRequestException	Invalid request / resourceInUse
403 Forbidden	403	ForbiddenException	Insufficient permissions
404 Not Found	404	NoSuchTable/NamespaceException	Resource not found
409 Conflict	409	AlreadyExistsException	Already exists
412 Precondition Failed	412	CommitFailedException	ETag mismatch (OCC)
500 Server Error	500	ServiceFailureException	Server error
503 Unavailable	503	ServiceUnavailableException	Temporarily unavailable

[Key design] HTTP client uses setThrowExceptionOnExecuteError(false), allowing inspection of responses for context-specific exception mapping (table vs namespace for 404).

9. Summary

The Iceberg BigQuery module is a compact yet well-engineered integration demonstrating key architectural principles:

- Clean Separation of Concerns: Each class has a single responsibility - Catalog handles lifecycle, TableOperations handles metadata, Client handles API, utilities handle config.
- Maximal Reuse of Iceberg Core: Extends BaseMetastoreCatalog and BaseMetastoreTableOperations, inheriting table loading, metadata management, metrics, and conflict detection.
- Robust Concurrency: ETag-based OCC ensures concurrent metadata updates are detected correctly, preventing data corruption in multi-writer scenarios.
- Production-Ready Resilience: Built-in retry with exponential backoff, comprehensive exception mapping, and careful cleanup in failure paths.
- Testability by Design: Client interface allows complete test isolation via FakeBigQueryMetastoreClient.
- Cross-Engine Compatibility: Hive-compatible parameters ensure tables work across Spark, Flink, Trino without modification.

Metric	Value
Total Source Files	6 (main) + 4 (test) = 10
Total Lines of Code	~1,691 (main)
Design Patterns	Template Method, Strategy, OCC, Builder/Factory, Retry
Iceberg SPIs	Catalog, SupportsNamespaces, TableOperations, Configurable
Dependencies	Google BigQuery Java API, Google Auth Library
Namespace Model	Single-level (BigQuery Dataset = Iceberg Namespace)
Concurrency Control	ETag-based OCC via HTTP If-Match header
Authentication	ADC + Service Account Impersonation